C8205-Lochstreifen-Hilfsprogramme

Lochstreifen-Hilfsprogramme · Stand 10.10.2026

Dieses PDF gibt die Quelldatei `C8205-Interpreter-Lochstreifen-Hilfsprogramme.md` im Wortlaut wieder (7 Abschnitte). Weich umbrochene Zeilen sind zu Absätzen zusammengeführt; Listen, Tabellen, Codeblöcke und Zitate entsprechen der Quelle. Maßgeblich bleibt die .md-Datei; dieses PDF wird nicht automatisch nachgeführt (Stand siehe oben und Fußzeile).

Inhalt

1. Zweck . . . 2

2. Format des virtuellen Lochstreifens . . . 3

3. Verfahren . . . 3

3.1 Anzeige (c8205_lochstreifen_anzeige.py) . . . 3

3.2 Eingabe (c8205_lochstreifen_eingabe.py) . . . 3

3.3 Kopieren in die Zwischenablage (beide Programme) . . . 4

4. Benutzung . . . 4

4.1 Anzeige . . . 4

4.2 Eingabe . . . 4

4.3 Typischer Ablauf . . . 5

5. Technische Voraussetzungen (Installation siehe 5a) . . . 5

5a. Installation der benötigten Programme . . . 6

Linux . . . 6

Windows . . . 6

macOS . . . 6

WSL (Windows-Subsystem für Linux) . . . 6

Virtuelle Maschinen (Abgleich der Zwischenablage mit dem Host) . . . 6

6. Test unter Windows (noch offen) . . . 7

Lochstreifen-Hilfsprogramme

Quelle: C8205-Interpreter-Lochstreifen-Hilfsprogramme.md · im Wortlaut der Quelle

Version: 0.03-003

Dokumentversion s. oben; **Programmversion 1.9** (Stand 10.10.2026). Die Programme tragen ihre eigene Versionsnummer (`--version`), unabhängig vom Schema `y.xx-zzz` der Dateien (`C8205-Versionierung.md`, Abschnitt 2).

Programm	Datei	Version
Lochstreifen ausschreiben (Anzeige)	`c8205_lochstreifen_anzeige.py`	1.9
Lochstreifen erzeugen (Eingabe)	`c8205_lochstreifen_eingabe.py`	1.9

Die Versionsnummer lässt sich mit `--version` abfragen.

Änderungen

- **1.9**: `Um. KB` / `Um. GB` werden in Anzeige und Eingabefenster **nicht mehr ausgegeben**, sie wirken nur (Zeichenlage). „Irrung Zeichen“ löscht das letzte Zeichen der Zeile, „Irrung Satz“ die ganze bisherige Zeile; beide erscheinen nicht als Marke. Zeilenumbrüche werden dadurch nie zurückgenommen. Auf dem Lochstreifen selbst bleiben alle Lochungen unverändert erhalten (Eingabe stanzt sie weiterhin).
- **1.8**: Unter X11 wird zuerst `xsel --clipboard --input` benutzt, `xclip` nur noch, wenn `xsel` fehlt oder scheitert. Grund: In einer VMware-Umgebung kam der Text von `xclip` und von Tk nicht in der VM an, der Weg über `xsel` (vim: `:w !xsel -b`) dagegen schon.
- **1.7**: Neue Option `--zwischenablage auto|tk|system` zur Wahl des Kopierverfahrens (Vorgabe `auto`). `tk` benutzt nur die Tk-Zwischenablage, `system` nur Systemprogramm bzw. Win32-API. Gedacht für Umgebungen mit eigenem Zwischenablage-Abgleich (z. B. VMware), in denen sich Verfahren unterschiedlich verhalten.
- **1.6**: `xclip` wird wieder mit `-t UTF8_STRING` aufgerufen. Ohne Angabe bietet `xclip` den Text nur als `STRING` an; auf einer virtuellen Maschine übernahm der Zwischenablage-Abgleich zum Host solche Inhalte nicht (Terminal-Kopieren kam an, `echo … | xclip -selection clipboard` nicht).
- **1.5**: Derselbe Text wird innerhalb von 5 Sekunden nicht erneut in die Zwischenablage gelegt (Maus loslassen und Strg+C kurz nacheinander). Grund: Auf virtuellen Maschinen verarbeitet das Gast-Zusatzprogramm für den Zwischenablage-Abgleich mit dem Host jeden Besitzerwechsel einzeln; viele Wechsel hintereinander können den Abgleich stören.
- **1.4**: Unter Linux/macOS/WSL wird zuerst das Systemprogramm benutzt (`xclip -selection clipboard -t UTF8_STRING`); die Tk-Zwischenablage ist nur noch Rückfall, wenn kein Programm vorhanden ist oder es scheitert. Grund: Tk und `xclip` kämpften nach einem Test um die Besitzerrolle der Zwischenablage, im Programm kam beim Einfügen nichts an, obwohl der Terminalaufruf funktionierte.
- **1.3**: (Der Aufruf von `xclip` mit `-t UTF8_STRING` wurde in 1.4 wieder zurückgenommen.) bei `--debug` folgt nach dem Kopieren eine Gegenprobe (`xclip -o`/`xsel`), die zeigt, was wirklich in der Zwischenablage liegt, samt angebotenen Zielen.
- **1.2**: Das Kopierverfahren wird **bei jedem Kopieren dynamisch** nach dem laufenden System gewählt (Windows: Win32-API; Linux/macOS/WSL: Tk **und zusätzlich** ein Systemprogramm wie `wl-copy`, `xclip`, `xsel`, `pbcopy`, `clip.exe`, sofern vorhanden). Hintergrund: Die reine Tk-Zwischenablage geht unter X11/Wayland (XWayland) zeitweise verloren. Die Titelleiste nennt das erfolgreiche Verfahren, z. B. „kopiert (12 Zeichen, Tk+xclip)“.
- **1.1**: Markieren und Kopieren in die Zwischenablage jetzt unter Linux und Windows (unter Windows über die Win32-API, mit Wiederholungsversuchen und Rückfall auf Tk). Strg+C, Strg+A und Strg+Einfg werden unter Windows auch über den Tastenkode erkannt. Eingabe: AltGr-Zeichen (`@ { } [ ] \ ~ |`) werden unter Windows nicht mehr als Strg-Befehl behandelt.
- **1.0**: Erste Fassung (unter Linux getestet).

> **Hinweis zum Teststand:** Version 1.2 ist unter Linux nur mit Attrappen der Systemprogramme geprüft, die Fenster selbst nicht automatisch. Der Windows-Zweig wurde noch **nicht** auf einem Windows-Rechner getestet (siehe Abschnitt „Test unter Windows“).

---

1. Zweck

Beide Programme sind Hilfsmittel zum C8205-Interpreter `c8205_interp.py`. Sie arbeiten mit dem **virtuellen Lochstreifen**, wie ihn der Interpreter an den Eingängen `--leser1` / `--leser2` liest und am Stanzer (`--stanzer1`) schreibt.

- **Anzeige** liest einen virtuellen Lochstreifen und schreibt ihn als lesbaren Text in einem scrollbaren Fenster aus (entspricht dem Fenster „SW Ausgabe“ von `--SW-TERMINAL` des Interpreters; beide Fenster bleiben bis zum Schließen mit X offen).
- **Eingabe** ist das Gegenstück: Sie erzeugt einen virtuellen Lochstreifen aus Tastatureingaben in einem Fenster.

Der Text in beiden Fenstern kann mit der Maus markiert und in die Zwischenablage kopiert werden.

2. Format des virtuellen Lochstreifens

- Eine Lochungszeile je Textzeile.
- 8 Zeichen `0`/`1` in der Spurfolge `w v u p 8 4 2 1` (`1` = gelocht).
- Leerzeilen werden beim Lesen übersprungen.
- Zeichenvorrat und Lochungen nach **Kodetabelle 1** (Spalte „Ausgabe-Adresse“), identisch zu `TABELLE1` in `c8205_interp.py`.
- Gültige Zeichen haben **ungerade** Parität (ungerade Zahl gelochter Spuren). Ausnahme: „Irrung Zeichen“ (alle 8 Spuren).

3. Verfahren

3.1 Anzeige (`c8205_lochstreifen_anzeige.py`)

1. Der Lochstreifen (Datei oder Verzeichnis) wird gelesen. Bei einem Verzeichnis werden alle Dateien ohne führenden Punkt in Namensreihenfolge hintereinander gelesen; zwischen den Dateien erscheint eine graue Trennzeile `--- Datei: NAME ---`.
2. Jede Zeile wird anhand der Kodetabelle in ein Zeichen übersetzt. Die **Zeichenlage** (Klbuch/Gbuch) wird durch die Zeichen `Um. KB` / `Um. GB` umgeschaltet und gilt auch über Dateigrenzen hinweg. Anfangslage ist KB (mit `--lage GB` änderbar).
3. Darstellung:
   - Normale Zeichen erscheinen als Text. SatzMarke/WRZL ergibt einen Zeilenumbruch, Tabulator einen Tabulator, Leertaste/ZR ein Leerzeichen.
   - `Um. KB`/`Um. GB` erscheinen nicht, sie schalten nur die Zeichenlage. `Irrung Zeichen` löscht das vorige Zeichen der Zeile, `Irrung Satz` die ganze Zeile. Sonstige Zeichen ohne druckbare Entsprechung erscheinen als **blaue Marke**, z. B. `<Transport (Relikt)>`.
   - Zeilen ohne Datenlochung (Vorspann, `00000000`) werden übersprungen.
   - Zeilen mit gerader Zahl gelochter Spuren (außer „Irrung Zeichen“) erscheinen als **rote Marke** `<Paritätsfehler 01010101>`; unbekannte Lochungen als `<unbekannte Lochung …>`; Zeilen, die keine 8 Zeichen `0`/`1` sind, als `<ungültige Zeile …>`.
4. Die Ausgabe erscheint in einem Tkinter-Fenster (130 × 36 Zeichen, Bildlaufleiste, Standardposition rechts unten), das bis zum Schließen mit X offen bleibt. Ohne Fenstermöglichkeit oder mit `--text` geht die Ausgabe auf das Terminal.

3.2 Eingabe (`c8205_lochstreifen_eingabe.py`)

1. Es öffnet sich ein gleichartiges Fenster. Jedes eingegebene Zeichen des R300-Kodes wird **sofort** als Lochungszeile in die Lochstreifendatei geschrieben. Eine vorhandene Datei wird überschrieben, fehlende Ordner werden angelegt.
2. Der Stanzer-Kode kennt die Zeichenlage nicht. Das Programm stanzt `Um. KB` bzw. `Um. GB` selbst, sobald ein Zeichen die andere Lage braucht (Anfang: KB), zeigt es aber nicht an.
3. Eine Lochung lässt sich nicht zurücknehmen (wie am echten Stanzer). Korrektur durch Stanzen von „Irrung Zeichen“ bzw. „Irrung Satz“; im Fenster verschwindet dabei das letzte Zeichen bzw. die Zeile.
4. Zeichen, die nicht im R300-Kode vorkommen, werden abgewiesen (Signalton, Hinweis in der Titelleiste); es wird nichts gestanzt.
5. Das Textfeld ist nicht editierbar; es zeigt nur, was gestanzt wurde.

3.3 Kopieren in die Zwischenablage (beide Programme)

Das Fenster „SW Ausgabe“ des Interpreters (`--SW-TERMINAL`, `c8205_interp.py` ab Version 0.03-007) kopiert auf dieselbe Weise; das Kopierverfahren wählt dort der Schalter `--zwischenablage auto|tk|system` (wie hier).

Kopiert wird

- sofort beim **Loslassen der Maustaste** nach dem Markieren (auch nach Doppelklick),
- mit **Strg+C**, **Strg+Umschalt+C** oder **Strg+Einfg** (auch wenn der Tastaturfokus nicht im Textfeld liegt),
- über das **Kontextmenü** (rechte Maustaste; auf macOS auch mittlere Taste bzw. Strg+Klick) mit „Kopieren“ und „Alles markieren“.

Strg+A markiert den gesamten Text. Jedes erfolgreiche Kopieren meldet sich kurz in der Titelleiste („kopiert (N Zeichen)“), ein Fehlschlag mit „Kopieren fehlgeschlagen“.

Technisch je nach System:

System	Verfahren
Linux, macOS, WSL	zuerst ein Systemprogramm, das die Zwischenablage dauerhaft übernimmt (Tk nur als Rückfall, `clipboard_clear`, `clipboard_append`, `update`, bis zu 5 Versuche): Wayland `wl-copy`, X11 `xsel` (sonst `xclip`), macOS `pbcopy`, WSL `clip.exe`. Welches, wird bei jedem Kopieren nach `WAYLAND_DISPLAY/XDG_SESSION_TYPE/DISPLAY` und den installierten Programmen entschieden; fehlende Programme werden übersprungen
Windows	Win32-API (`user32`/`kernel32`, Format `CF_UNICODETEXT`, Zeilenenden `\r\n`), bis zu 20 Versuche im Abstand von 50 ms, falls ein anderes Programm die Zwischenablage kurz sperrt; bei Fehlschlag Rückfall auf die Tk-Zwischenablage

Unter Windows werden Strg+C, Strg+A und Strg+Einfg zusätzlich über den Tastenkode erkannt, damit sie auch bei nicht-lateinischem Tastaturlayout wirken.

Nur die **Eingabe** kann außerdem einfügen: Strg+V oder Umschalt+Einfg stanzt die Zeichen aus der Zwischenablage.

4. Benutzung

4.1 Anzeige

```
python3 c8205_lochstreifen_anzeige.py --LS-LESER PFAD [--lage KB|GB] [--text] [--debug] [--version]
```

Option	Bedeutung
`--LS-LESER PFAD`	**Pflicht.** Virtueller Lochstreifen: Datei oder Verzeichnis
`--lage KB|GB`	Zeichenlage am Anfang (Vorgabe `KB`)
`--text`	Ausgabe auf dem Terminal statt im Fenster (auch Rückfall, wenn kein Fenster möglich ist)
`--zwischenablage auto|tk|system`	Kopierverfahren (Vorgabe `auto`): `tk` nur Tk-Zwischenablage, `system` nur Systemprogramm (`xclip` usw.) bzw. Win32-API
`--debug`	Diagnose auf der Fehlerausgabe: Tasten, Mausereignisse, Kopiervorgänge (inkl. verwendetes Verfahren `Win32`/`Tk`)
`--version`	Versionsnummer ausgeben

Beispiele:

```
python3 c8205_lochstreifen_anzeige.py --LS-LESER stanzer-ausgabe.txt
python3 c8205_lochstreifen_anzeige.py --LS-LESER lochstreifen/ --lage GB
python3 c8205_lochstreifen_anzeige.py --LS-LESER band.txt --text > band_klartext.txt
```

Rückgabewert 2 bei einem Lese- oder Dateifehler.

4.2 Eingabe

```
python3 c8205_lochstreifen_eingabe.py --LS-STANZER PFAD [--text] [--debug] [--version]
```

Option	Bedeutung
`--LS-STANZER PFAD`	**Pflicht.** Zu erzeugender Lochstreifen. Datei, oder Verzeichnis (bzw. Pfad mit `/` am Ende): dann `VERZ/stanzer-ausgabe.txt`
`--text`	Zeichen von der Standardeingabe lesen statt im Fenster (auch Rückfall); Ende mit Strg+D (Windows: Strg+Z, Enter)
`--zwischenablage auto|tk|system`	Kopierverfahren wie bei der Anzeige
`--debug`	Diagnose auf der Fehlerausgabe
`--version`	Versionsnummer ausgeben

Beispiele:

```
python3 c8205_lochstreifen_eingabe.py --LS-STANZER eingabe.txt
python3 c8205_lochstreifen_eingabe.py --LS-STANZER lochstreifen/
echo "hallo" | python3 c8205_lochstreifen_eingabe.py --LS-STANZER eingabe.txt --text
```

Unter Windows ggf. `py` oder `python` statt `python3` verwenden.

Tasten im Eingabefenster:

Taste	Wirkung
Buchstaben, Ziffern, Satzzeichen des R300-Kodes	Zeichen stanzen (ggf. mit vorausgehendem `Um. KB`/`Um. GB`)
Leertaste	Leertaste/ZR
Tab	Tabulator
Enter	SatzMarke/WRZL
F1 / F2	`Um. KB` / `Um. GB` von Hand stanzen
Rück-Taste	„Irrung Zeichen“ (alle 8 Spuren)
Entf	„Irrung Satz“
Strg+V, Umschalt+Einfg	Zwischenablage stanzen
Strg+C, Strg+Einfg, Maus loslassen	Markierten Text kopieren

Tippbare Sonderzeichen: Klbuch-Lage `% / + : . , -`, der Gravis `` ` `` sowie Ziffern; Gbuch-Lage `< " [ > ' ~ ] # * ( = ; ) ? !` und die Glyphen ≈ ≋ △.

4.3 Typischer Ablauf

1. Programm mit dem Interpreter ausführen, der über `--stanzer1 band.txt` einen Lochstreifen schreibt.
2. Mit der Anzeige ansehen: `--LS-LESER band.txt`.
3. Oder mit der Eingabe einen Lochstreifen erzeugen und dem Interpreter mit `--leser1 eingabe.txt` übergeben.

5. Technische Voraussetzungen (Installation siehe 5a)

- **Python 3** (getestet mit Python 3; nötig sind f-Strings und `argparse`, daher mindestens 3.6).
- Nur **Standardbibliothek**: `argparse`, `os`, `sys`, `tkinter`, außerdem `ctypes` und `time` für die Windows-Zwischenablage. Es ist nichts nachzuinstallieren.
- **Tkinter** für die Fenster:
  - Linux: Paket `python3-tk` (z. B. `sudo apt install python3-tk`).
  - Windows und macOS: im Python-Installer von python.org enthalten.
- **Grafische Oberfläche**: unter Linux muss `DISPLAY` oder `WAYLAND_DISPLAY` gesetzt sein. Fehlt sie oder fehlt Tkinter, geben beide Programme eine Warnung aus und weichen auf den Textmodus (`--text`) aus.
- **Zeichenkodierung**: Lochstreifendateien sind UTF-8; die Standardausgabe der Anzeige wird auf UTF-8 umgestellt.
- **Virtuelle Maschine:** Der Abgleich der Zwischenablage mit dem Host braucht das Gast-Zusatzprogramm (VirtualBox: Gasterweiterungen, QEMU/SPICE: `spice-vdagent`, VMware: `open-vm-tools`). Das Programm legt den Text nur in der Zwischenablage des Gastes ab; wie sie zum Host kommt, entscheidet dieses Zusatzprogramm.

- **Optional unter Linux**, für zuverlässiges Kopieren: Wayland `wl-clipboard` (`sudo apt install wl-clipboard`), X11 `xsel` (`sudo apt install xsel`) oder `xclip` (`sudo apt install xclip`). Ohne diese Programme wird nur die Tk-Zwischenablage verwendet.
- **Schreibrechte** für den Zielpfad (nur Eingabe).
- Betriebssysteme: Linux (getestet), Windows (Zwischenablage-Zweig noch ungetestet), macOS (nicht getestet, Tk-Verfahren wie unter Linux).

5a. Installation der benötigten Programme

Benötigt wird nur Python 3 mit Tkinter. Die Zwischenablage-Programme sind optional (nur Linux/WSL). Die beiden `.py`-Dateien selbst müssen nicht installiert werden, sie laufen direkt.

Linux

Python 3 ist meist vorhanden (Prüfung: `python3 --version`).

Distribution	Python und Tkinter	Zwischenablage (optional)
Debian, Ubuntu, Mint	`sudo apt install python3 python3-tk`	X11: `sudo apt install xsel xclip`; Wayland: `sudo apt install wl-clipboard`
Fedora	`sudo dnf install python3 python3-tkinter`	X11: `sudo dnf install xsel xclip`; Wayland: `sudo dnf install wl-clipboard`
Arch, Manjaro	`sudo pacman -S python tk`	X11: `sudo pacman -S xsel xclip`; Wayland: `sudo pacman -S wl-clipboard`
openSUSE	`sudo zypper install python3 python3-tk`	X11: `sudo zypper install xsel xclip`; Wayland: `sudo zypper install wl-clipboard`

Prüfung, ob Tkinter da ist: `python3 -m tkinter` öffnet ein kleines Testfenster. Ob X11 oder Wayland läuft, zeigt `echo $XDG_SESSION_TYPE`.

Windows

1. Python 3 von <https://www.python.org/downloads/> herunterladen und installieren. Im Installer **„Add python.exe to PATH“** ankreuzen. Die Option **„tcl/tk and IDLE“** ist vorgegeben und muss angekreuzt bleiben.
2. Prüfung in der Eingabeaufforderung oder PowerShell: `py --version` und `py -m tkinter` (öffnet ein Testfenster).
3. Aufruf mit `py c8205_lochstreifen_anzeige.py --LS-LESER band.txt`.
4. Weitere Programme sind nicht nötig: Die Zwischenablage läuft über die Win32-API, `ctypes` gehört zu Python.

Hinweis: Die Python-Version aus dem Microsoft Store enthält ebenfalls Tkinter. Bei einer abgespeckten Installation ohne Tcl/Tk Python über „Ändern“ im Installer nachrüsten.

macOS

Python 3 von <https://www.python.org/downloads/> installieren (enthält Tkinter), Aufruf mit `python3`. `pbcopy` ist im System vorhanden. Die Python-Version von Homebrew braucht zusätzlich `brew install python-tk`.

WSL (Windows-Subsystem für Linux)

Wie Linux (Debian/Ubuntu: `sudo apt install python3 python3-tk`). Fenster brauchen WSLg (Windows 11 oder aktuelles Windows 10). Die Zwischenablage zum Windows-Host geht über `clip.exe`, das in WSL schon vorhanden ist.

Virtuelle Maschinen (Abgleich der Zwischenablage mit dem Host)

Umgebung	Installation im Gast (Debian/Ubuntu)
VMware	`sudo apt install open-vm-tools open-vm-tools-desktop`, danach Neustart
VirtualBox	Gasterweiterungen aus dem Menü „Geräte“ einlegen und installieren; Zwischenablage: Geräte, Gemeinsame Zwischenablage, bidirektional
QEMU/SPICE	`sudo apt install spice-vdagent`

6. Test unter Windows (noch offen)

Zu prüfen, sobald ein Windows-Rechner verfügbar ist:

1. Anzeige mit einer beliebigen Lochstreifendatei starten, Text mit der Maus markieren und in einem anderen Programm (z. B. Editor) einfügen.
2. Dasselbe mit Strg+C, Strg+Einfg und dem Kontextmenü.
3. Im Eingabefenster `@`, `{`, `}`, `[`, `]`, `\`, `~`, `|` per AltGr tippen (soweit im R300-Kode vorhanden, sonst Signalton) und prüfen, dass sie nicht als Steuerbefehl wirken.
4. Bei Problemen mit `--debug` starten und die Ausgabe auswerten; dort steht, ob das Kopieren per `Win32` oder per `Tk` lief.